Java: Cryptocurrency Price Management System

Hard

Overview

Implement a system to manage cryptocurrency prices using the Observer Design Pattern. The system should allow for concurrent updates and maintain a history of all price changes for each cryptocurrency. Implement the CryptoCurrencyListenerImpl and CryptoCurrencyManager classes. In the main method of the Solution class, an ExecutorService spawns multiple threads concurrently executing the executeCommand method. This method updates cryptocurrency prices based on user commands, timestamping each action. Three instances of CryptoCurrencyListenerImpl are instantiated for BTC, ETH, and DOGECOIN, registered with the CryptoCurrencyManager using the registerListener method. After processing all commands, the program outputs the current prices for each currency via the getPriceDescription method and command histories via the getCommandsHistory method. The executeCommand method handles commands to update cryptocurrency prices. It extracts the cryptoCurrency code, action ("increase" or "decrease"), and amount from the input command string, then submits these details to the CryptoCurrencyManager instance for concurrent execution, timestamped with the current time. The application will receive a list of string commands[n], where each string represents a command for managing cryptocurrency prices. Each command consists of a cryptoCurrency code (BTC, ETH, or DOGECOIN), an action ("increase" or "decrease"), and an amount, separated by spaces. Example Input: Output: Explanation: Command Class Description Complete the class CryptoCurrencyListenerImpl in the editor below. Methods Complete the class CryptoCurrencyManager in the editor below. Methods Constraints The first line contains an integer n, the size of the array commands. Each of the next n lines contains 3 words separated by space, denoting commands[i]. Sample Input 0 Sample Output 0 Explanation Sample Input 1 Sample Output 1 Explanation The CryptoCurrencyListenerImpl class implements the CryptoCurrencyListener interface (provided in the editor), maintaining the current price and a history of all price changes for a cryptocurrency. * It provides methods to increase or decrease the price, update the price based on an action, and retrieve the currency code, current price, and command history. The CryptoCurrencyManager class manages a collection of cryptocurrency listeners and facilitates concurrent notification of price updates to registered listeners.Three instances of CryptoCurrencyListenerImpl are registered for BTC, ETH, and DOGECOIN using the registerListener method of CryptoCurrencyManager.For each command[i], a task is submitted to the ExecutorService to execute the executeCommand method concurrently. Here is one possible way of executing the program:After processing these commands, getPriceDescription and getCommandsHistory methods were called on each CryptoCurrencyListenerImpl instance, and their results were returned. The results of the getCommandsHistory method were sorted based on their timestamps before being returned. CryptoCurrencyListenerImpl(String cryptoCurrency): Initializes the instance with the specified cryptoCurrency code and sets the initial price to 0, preparing to record price changes. increase(int amount): Increases the cryptocurrency's price by the specified amount. decrease(int amount): Decreases the cryptocurrency's price by the specified amount. updatePrice(String action, int amount, LocalDateTime currentTime): Updates the cryptocurrency's price based on the specified action ("increase" or "decrease") and amount and records the change with a currentTime timestamp. getCurrencyCode() -> String: Returns the cryptocurrency's code. getPrice() -> Integer: Returns the current price of the cryptocurrency. getCommandsHistory() -> List<String>: Returns a list of all recorded price change commands, sorted by the timestamp. registerListener(CryptoCurrencyListener listener): Registers a new cryptocurrency listener to receive price update notifications. updatePrice(String cryptoCurrency, String action, int amount, LocalDateTime currentTime): Updates the price of the cryptocurrency by notifying all registered listeners with the given action ("increase" or "decrease"), amount, and currentTime timestamp.$1 \leq n \leq 10$, where n is the size of the array commands $0 < amount \leq 1000$Possible cryptoCurrency code values: "BTC", "ETH", and "DOGECOIN".Possible action code values: "increase" and "decrease".Three instances of CryptoCurrencyListenerImpl are registered for BTC, ETH, and DOGECOIN using the registerListener method of CryptoCurrencyManager.For each command[i], a task is submitted to the ExecutorService to execute the executeCommand method of the Result class concurrently. Here is one possible way of executing the program:After processing these commands, getPriceDescription and getCommandsHistory methods were called on each

CryptoCurrencyListenerImpl instance, and their results were returned. The results of the getCommandsHistory method were sorted based on their timestamps before being returned.Three instances of CryptoCurrencyListenerImpl are registered for BTC, ETH, and DOGECOIN using the registerListener method of CryptoCurrencyManager.For each command[i], a task is submitted to the ExecutorService to execute the executeCommand method of the Result class concurrently. Here is one possible way of executing the program: Command Action BTC increase 100 Increases BTC price by 100. The current price is 100. Commands history (unsorted): [increase 100]. BTC decrease 50 Decreases BTC price by 50. The current price is 50. Commands history (unsorted): [increase 50, decrease 50]. BTC decrease 50 Decreases BTC price by 50. The current price is 0. Commands history (unsorted): [increase 50, decrease 50, decrease 50]. DOGECOIN increase 100 Increases DOGECOIN price by 100. The current price: 100. Commands history (unsorted): [increase 100]. After processing these commands, getPriceDescription and getCommandsHistory methods were called on each CryptoCurrencyListenerImpl instance, and their results were returned. The results of the getCommandsHistory method were sorted based on their timestamps before being returned.
Java (Advanced)
Coding
100 score
40 mins
15 test cases
Java, Problem Solving, MultiThread, Sorting, Synchronization

Question

Implement a system to manage cryptocurrency prices using the Observer Design Pattern. The system should allow for concurrent updates and maintain a history of all price changes for each cryptocurrency.

Implement the *CryptoCurrencyListenerImpl* and *CryptoCurrencyManager* classes.

- The *CryptoCurrencyListenerImpl* class implements the *CryptoCurrencyListener* interface (provided in the editor), maintaining the current price and a history of all price changes for a cryptocurrency.
    - It provides methods to increase or decrease the price, update the price based on an action, and retrieve the currency code, current price, and command history.
- The *CryptoCurrencyManager* class manages a collection of cryptocurrency listeners and facilitates concurrent notification of price updates to registered listeners.

In the *main* method of the *Solution* class, an *ExecutorService* spawns multiple threads concurrently executing the *executeCommand* method. This method updates cryptocurrency prices based on user commands, timestamping each action.

Three instances of *CryptoCurrencyListenerImpl* are instantiated for BTC, ETH, and DOGECOIN, registered with the *CryptoCurrencyManager* using the *registerListener* method. After processing all commands, the program outputs the current prices for each currency via the *getPriceDescription* method and command histories via the *getCommandsHistory* method.

The *executeCommand* method handles commands to update cryptocurrency prices. It extracts the *cryptoCurrency* code, *action* ("increase" or "decrease"), and amount from the input command string, then submits these details to the *CryptoCurrencyManager* instance for concurrent execution, timestamped with the current time.

The application will receive a list of string *commands[n]*, where each string represents a command for managing cryptocurrency prices. Each command consists of a

cryptoCurrency code (BTC, ETH, or DOGECOIN), an *action* ("increase" or "decrease"), and an *amount*, separated by spaces.

Example

Input:

```
BTC increase 100
BTC increase 200
BTC decrease 50
BTC increase 100
```

Output:

```
Current Price : BTC=350
History : [increase 100, increase 200, decrease 50, increase 100]
Current Price : ETH=0
History : []
Current Price : DOGECOIN=0
History : []
```

Explanation:

- Three instances of *CryptoCurrencyListenerImpl* are registered for BTC, ETH, and DOGECOIN using the *registerListener* method of *CryptoCurrencyManager*.
- For each *command[i]*, a task is submitted to the *ExecutorService* to execute the *executeCommand* method concurrently. Here is one possible way of executing the program:

Comma	Action
BTC increas	Increases BTC price by 200. The current price is 200. Commands history (unsorted): [increase 200].
BTC	Decreases BTC price by 50. The current price is 150. Commands history
BTC increas	Increases BTC price by 100. The current price is 250. Commands history (unsorted): [increase 200, decrease 50, increase 100].
BTC increas	Increases BTC price by 100. The current price is 350. Commands history (unsorted): [increase 200, decrease 50, increase 100, increase 100].

- After processing these commands, *getPriceDescription* and *getCommandsHistory* methods were called on each *CryptoCurrencyListenerImpl* instance, and their results were returned. The results of the *getCommandsHistory* method were sorted based on their timestamps before being returned.

Class Description

Complete the class *CryptoCurrencyListenerImpl* in the editor below.

Methods

- *CryptoCurrencyListenerImpl*(String *cryptoCurrency*): Initializes the instance with the specified *cryptoCurrency* code and sets the initial price to 0, preparing to record price changes.
- *increase*(int *amount*): Increases the cryptocurrency's price by the specified *amount*.
- *decrease*(int *amount*): Decreases the cryptocurrency's price by the specified *amount*.
- *updatePrice*(String *action*, int *amount*, LocalDateTime *currentTime*): Updates the cryptocurrency's price based on the specified *action* ("increase" or "decrease") and *amount* and records the change with a *currentTime* timestamp.
- *getCurrencyCode*() -> String: Returns the cryptocurrency's code.
- *getPrice*() -> Integer: Returns the current price of the cryptocurrency.
- *getCommandsHistory*() -> List<String>: Returns a list of all recorded price change commands, sorted by the timestamp.

Complete the class *CryptoCurrencyManager* in the editor below.

Methods

- *registerListener*(CryptoCurrencyListener *listener*): Registers a new cryptocurrency *listener* to receive price update notifications.
- *updatePrice*(String *cryptoCurrency*, String *action*, int *amount*, LocalDateTime *currentTime*): Updates the price of the *cryptocurrency* by notifying all registered listeners with the given *action* ("increase" or "decrease"), *amount*, and *currentTime* timestamp.

Constraints

- $1 \le n \le 10$, where n is the size of the array *commands*
- $0 < amount \le 1000$
- Possible *cryptoCurrency* code values: "BTC", "ETH", and "DOGECOIN".
- Possible *action* code values: "increase" and "decrease".

Input Format for Custom Testing
The first line contains an integer *n*, the size of the array *commands*.
Each of the next *n* lines contains 3 words separated by space, denoting *commands[i]*.
Sample Case 0

Sample Input 0

```
4
BTC increase 100
BTC increase 200
ETH increase 50
DOGECOIN increase 100
```

Sample Output 0

```
Current Price : BTC=300
History : [increase 100, increase 200]
```

```
Current Price : ETH=50
History : [increase 50]
Current Price : DOGECOIN=100
History : [increase 50]
```

Explanation

- Three instances of *CryptoCurrencyListenerImpl* are registered for BTC, ETH, and DOGECOIN using the *registerListener* method of *CryptoCurrencyManager*.
- For each *command[i]*, a task is submitted to the *ExecutorService* to execute the *executeCommand* method of the *Result* class concurrently. Here is one possible way of executing the program:

Command	Action
BTC increase	Increases BTC price by 100. The current price is 100. Commands
ETH increase	Increases ETH price by 50. The current price is 50. Commands
DOGECOIN	Increases DOGECOIN price by 100. The current price is 100.
BTC increase	Increases BTC price by 200. The current price: 300. Commands

- After processing these commands, *getPriceDescription* and *getCommandsHistory* methods were called on each *CryptoCurrencyListenerImpl* instance, and their results were returned. The results of the *getCommandsHistory* method were sorted based on their timestamps before being returned.

Sample Case 1

Sample Input 1

```
4
BTC increase 100
BTC decrease 50
BTC decrease 50
DOGECOIN increase 100
```

Sample Output 1

```
Current Price : BTC=0
History : [increase 100, decrease 50, decrease 50]
Current Price : ETH=0
History : []
Current Price : DOGECOIN=100
History : [increase 100]
```

Explanation

- Three instances of *CryptoCurrencyListenerImpl* are registered for BTC, ETH, and DOGECOIN using the *registerListener* method of *CryptoCurrencyManager*.
- For each *command[i]*, a task is submitted to the *ExecutorService* to execute the *executeCommand* method of the *Result* class concurrently. Here is one possible way of executing the program:

Command	Action
BTC	Increases BTC price by 100. The current price is 100.
BTC	Decreases BTC price by 50. The current price is 50. Commands
BTC	Decreases BTC price by 50. The current price is 0. Commands
DOGECOIN	Increases DOGECOIN price by 100. The current price: 100.

- After processing these commands, *getPriceDescription* and *getCommandsHistory* methods were called on each *CryptoCurrencyListenerImpl* instance, and their results were returned. The results of the *getCommandsHistory* method were sorted based on their timestamps before being returned.